ML4SD

Lesson ③: Linear Models for Supervised Learning

1. Building blocks of ML pipeline

① Question

② Data $\{x^{(i)}\}_{i=1}^{n}$

$x^{(i)} \in \mathbb{R}^d$ ith element of the set

Unstructured

③ Embed them in a space $\phi(x)$: $\{\phi^{(i)}\}_{i=1}^{n}$

$\underbrace{\phi^{(i)}}_{\text{feature vector}} \in \mathbb{R}^{d'}$

↳ Structured numerical representation of the data

④ Model : $\underbrace{f_\theta(\phi^{(i)})}_{}\ (= \hat{y}^{(i)})$

function taking $\phi^{(i)}$ as input and giving back a prediction

$\theta \in \mathbb{R}^p$ are the parameters

⑤ An error function :

$R(D, \theta)$ scalar $D = \{x^{(i)}\}_{i=1}^{n}$

↑ training set ↑ parameters

↳ measures the error that the model f_θ is making on dataset D.

Generalization phase :

Test set $\tilde{X} = \{\tilde{x}^{(i)}\}_{i=1}^{n_{test}}$

$R_{test} = R(\tilde{D}, \hat{\theta})$

↑ best parameters obtained during training.

II - Supervised Learning

1. Setting

$$D = \{(\overset{\text{data}}{x^{(i)}}, \overset{\text{target}}{y^{(i)}})\}$$

$$(x^{(i)}, y^{(i)}) \in \mathbb{R}^d \times \mathcal{Y}$$

$\mathcal{Y}$ is continuous : Regression

for instance : temperature prediction ($\mathcal{Y} = \mathbb{R}$)

$\mathcal{Y}$ is discrete : Classification

for instance : $\mathcal{Y} = \{\text{'dog'}, \text{'cat'}\}$

2. Loss, risk and empirical risk

Ideally, the training aims at minimizing the expected risk :

$$(1) \quad R = \boxed{\mathbb{E}}\left[\ell(\overset{\text{truth}}{y}, \hat{y})\right] \qquad \hat{y} = f_\theta(x)$$

average

↑ estimate

loss function : measures the error of the model on a single example.

$$R = \int \ell(y, \hat{y}) \underbrace{p(y, \hat{y})}_{\text{unknown!}} dy\, d\hat{y}$$

$$\Rightarrow \boxed{\hat{R} = \frac{1}{n} \sum_{i=1}^{n} \overset{\text{loss}}{\ell}(y^{(i)}, \hat{y}^{(i)})} \quad (2)$$

truth or target for (i)

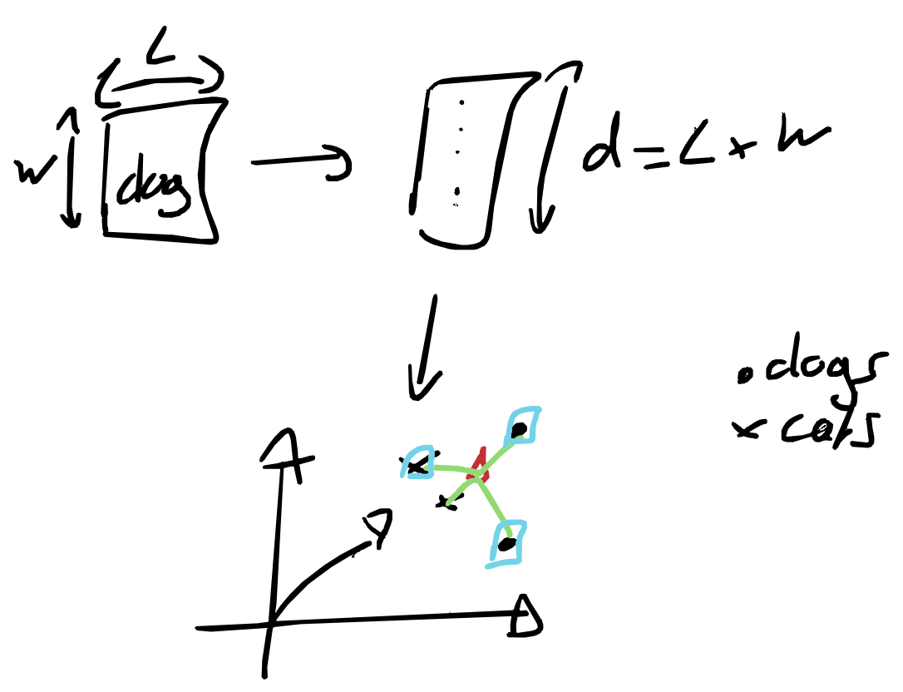
estimated ith element $f_\theta(x^{(i)}) = \hat{y}^{(i)}$

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\ \hat{R}(\theta)$$

Remark: With a finite number n of samples $\hat{R}(\theta) < R(\theta)$

This is why a test set $\tilde{x}$ is also important: unbiased estimate of $R(\theta)$.

3. Why do we need ML ?



$$R_{train} = \hat{R}(D) = 0$$

⇒ R_{test} is very large!

Why ?

Curse of dimensionality!

d in images is $\sim 10^3 - 10^6$

III - Linear Models for Regression

Remark: x is a feature vector

Ex: Predict salary y from one feature :

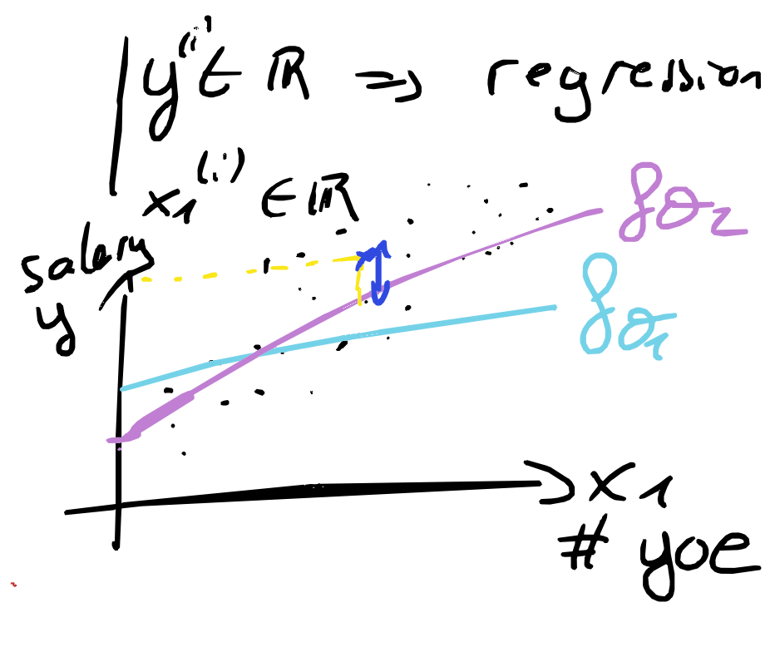
- x_1 : the # of years of experience

$n = 373$ pairs $(x_1^{(i)}, y^{(i)})$

↳ Supervised learning

$y^{(i)} \in \mathbb{R}$ ⇒ regression

$x_1^{(i)} \in \mathbb{R}$



Definition: The Linear regression model :

$$\hat{y}^{(i)} = f_\theta(x^{(i)})$$

$$= \theta_0 + \theta_1 x_1^{(i)} + \theta_2 x_2^{(i)} + \dots$$

features

parameters

$$= \underset{\text{intercept}}{\theta_0} + \overset{\text{slope}}{\theta_1} x_1^{(i)} \quad \text{in this case}$$

Geometrical picture. $d = 1$ ⇒ straight line. d-dim, hyperplan.

We have $\left\{\begin{matrix}\text{the model} \\ \text{data}\end{matrix}\right\}$, we need an error function

$$\boxed{\ell(y^{(i)}, \hat{y}^{(i)}) = (\hat{y}^{(i)} - y^{(i)})^2} \quad \text{squared error}$$

N.B. This a choice : an other one could have been

$$\ell(y^{(i)}, \hat{y}^{(i)}) = |\hat{y}^{(i)} - y^{(i)}|$$

So empirical risk minimization (ERM).

$$\underset{\theta}{\operatorname{argmin}}\ \hat{R}(D, \theta) = \underset{\theta}{\operatorname{argmin}}\ \frac{1}{n}\sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2$$

$$\boxed{\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\ \frac{1}{n}\sum_{i=1}^{n} (\underbrace{\theta_0 + \theta_1 x_1^{(i)}}_{} - y^{(i)})^2}$$

Remark: The model is "linear" wrt the parameters of the function, not the feature

How do we find $\hat{\theta}$?

$$\begin{cases} \dfrac{\partial \hat{R}}{\partial \theta_0} = 0 = \dfrac{2}{n}\displaystyle\sum_{i=1}^{n}(\theta_0 + \theta_1 x_1^{(i)} - y^{(i)}) \\ \dfrac{\partial \hat{R}}{\partial \theta_1} = 0 = \dfrac{2}{n}\displaystyle\sum_{i=1}^{n}(\theta_0 + \theta_1 x_1^{(i)} - y^{(i)})\, x_1^{(i)} \end{cases}$$

$$\theta_1 = \frac{\overline{xy} - \bar{x}\bar{y}}{\overline{x^2} - (\bar{x})^2} \quad \left(= \frac{\operatorname{cov}(x, y)}{\operatorname{var}(x)}\right)$$

In matrix form

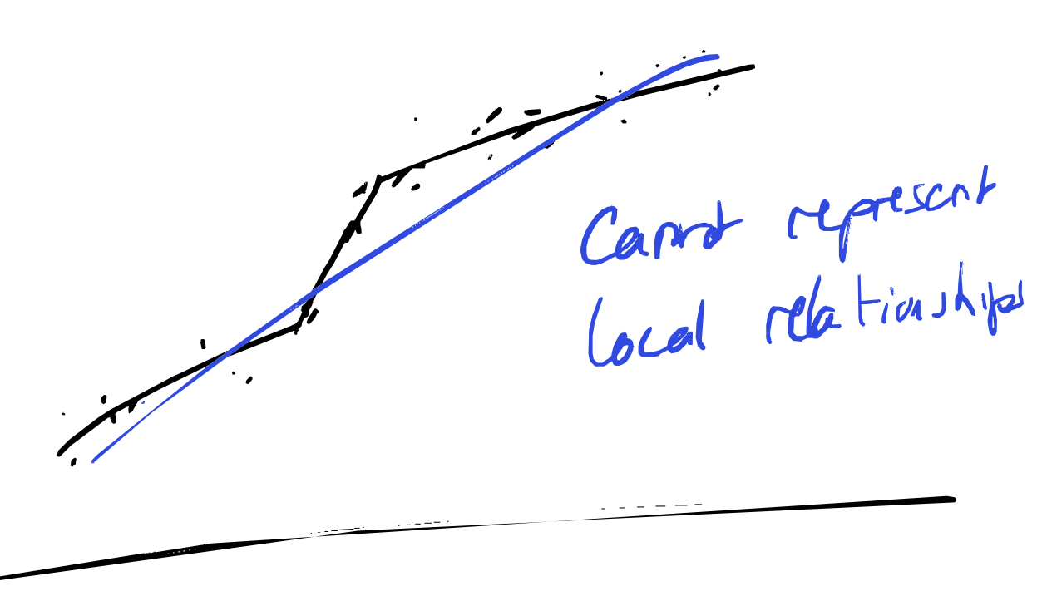
$$X = \begin{pmatrix} 1 & x_1^{(1)} & \dots & x_d^{(1)} \\ \vdots & \vdots & & \vdots \\ 1 & \underbrace{x_1^{(n)}}_{d} & & x_d^{(n)} \end{pmatrix} \in \mathbb{R}^{n \times (d+1)}$$

Feature matrix

$$y = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{pmatrix} \in \mathbb{R}^n$$

$$\hat{R}(\theta) = \frac{1}{n} \|X\theta - y\|_2^2$$

$$\boxed{\hookrightarrow \hat{\theta} = (X^T X)^{-1} X^T y}$$



IV - Linear Classification

1. Setting & optimal rule

$\mathcal{Y}$ discrete

Ex : Iris dataset with two features :

$n = 150$ couples $(x^{(i)}, y^{(i)})$

↳ supervised learning

$\begin{cases} x_1 : \text{sepal length} \\ x_2 : \text{sepal width} \end{cases}$ $x^{(i)} = [x_1^{(i)}, x_2^{(i)}]$

$y^{(i)} \in \{0, 1, 2\}$ 3 categories

3-class classification

Natural error function ?

0-1 loss :

$$\ell(y, \hat{y}) = \begin{cases} 1 & \text{if } \hat{y} \neq y \\ 0 & \text{otherwise} \end{cases}$$

What decision rule would you come with if you had access to $p(y = k | x)$

$$\hat{y}^{(i)} = \underset{k}{\operatorname{argmax}}\ p(y = k | x)$$

Bayes-optimal classifier

⚠ But: we don't know $p(y = k | x)$.

⇒ We need to model it!

2. The softmax model

$$\log\left[P_\theta(y^{(i)} = k | x^{(i)})\right] = \overbrace{\theta_k^T x^{(i)}}^{\text{linear part}} - \overbrace{\log Z}^{\text{normalizat.}}$$

Z ensures normalization (sum to 1)

$$P_\theta(y = k | x) = \frac{e^{\theta_k^T x}}{Z}$$

Imposing $\sum_k P_\theta(y = k | x) = 1$

gives $Z = \sum_{j=1}^{K} e^{\theta_j^T x}$

$$\boxed{P_\theta(y = k | x) = \frac{e^{\theta_k^T x}}{\sum_{j=1}^{K} e^{\theta_j^T x}}}$$

↳ The softmax model

Decision rule.

$$\hat{y} = \underset{k}{\operatorname{argmax}}\ P_\theta(y = k | x)$$

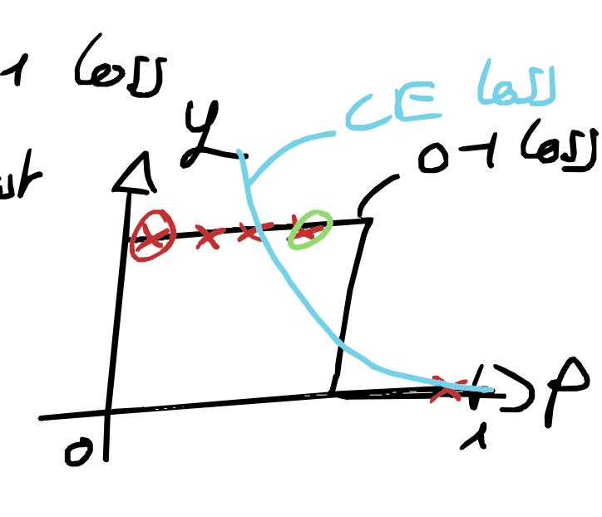
$$\boxed{\hat{y} = \underset{k}{\operatorname{argmax}}\ \underbrace{\theta_k^T x}_{\text{dot product between } \theta_k \text{ and } x}}$$

Geometrically. Each class has a vector representative θ_k and we assign to x the class with the largest overlap (dot product).

3. Learning (Fitting parameters)

Problem : 0-1 loss

0 gradient almost everywhere and not differentiable



Solution : A smooth surrogate of the 0-1 loss

$$\ell(y, \hat{y}) = -\overbrace{\sum_{k=0}^{K-1} \mathbb{1}_{y=k}}^{\text{selector}}\ \overbrace{\log P_\theta(y = k | x)}^{\text{output of the model}}$$

↳ The $\boxed{\text{cross-entropy}}$ loss

$\mathbb{1}_{y=k}$ is 1 if $y = k$ and 0 otherwise

So then to find $\hat{\theta}$ the best possible parameters

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}}\ \hat{R}(D, \theta)$$

$$= \underset{\theta}{\operatorname{argmin}}\ \frac{1}{n}\sum_{i=1}^{n} \ell(y^{(i)}, \hat{y}^{(i)})$$

$$= \underset{\theta}{\operatorname{argmin}}\ -\frac{1}{n}\sum_{i=1}^{n}\sum_{k=0}^{K-1} \mathbb{1}_{y=k} \log P_\theta(y = k | x)$$

⚠ Here, you cannot solve for θ analytically and we need numerical methods (Gradient descent)